RAG for Developers

From Basics to Advanced

1. What is RAG ?

- RAG (Retrieval-Augmented Generation) is a technique that combines information retrieval with LLMs to generate more accurate and up-to-date responses.
- Instead of relying only on the model's internal knowledge, it retrieves relevant information from external sources (like your documents, databases, websites, etc.) and uses that context to generate an answer.

Key Idea :
Give the model relevant context from your own data, so it can generate better and reliable answers.

2. Why is RAG Needed ?

- LLMs are trained on historical data and have a knowledge cutoff.
- They may not know your latest data, internal documents or domain-specific information.
- RAG helps in getting accurate, up-to-date and source-grounded answers.

3. LLM vs RAG

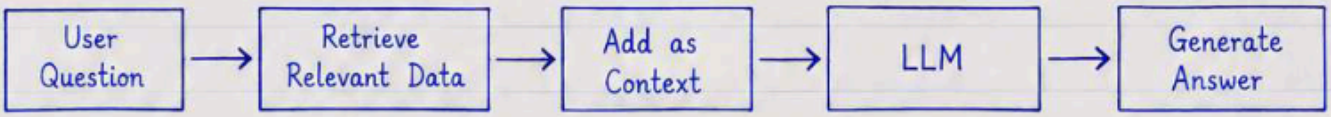
LLM (Alone)

- Uses only pre-trained knowledge
- May give outdated or incorrect answers
- Cannot access your private data
- Prone to hallucinations

RAG (With External Data)

- Uses external data (your documents, DB, etc.)
- Gives more accurate and relevant answers
- Can work with latest and private data
- Reduces hallucinations
- Provides source-grounded responses

4. Simple RAG Workflow



5. Real-world Example

- You have a company policy document.
- A user asks: "What is our leave policy?"
- RAG retrieves the relevant section from the document and passes it to the LLM, which then generates a precise answer.

Result :
Accurate, up-to-date and context-aware answer based on your documents.

RAG for Developers

From Basics to Advanced

RAG Architecture & How It Works

1. High Level Architecture

Documents → **Chunking** → **Embeddings** → **Vector DB**

- Documents
  - PDFs
  - Web pages
  - Database
  - Notion, etc.
- Chunking
  - Split text
  - Create chunks (with overlap)
- Embeddings
  - Convert chunks to vectors (using embedding model)
- Vector DB
  - Store vectors with metadata (Pinecone, Chroma, Weaviate, etc.)

User Query → **Retrieve Relevant Chunks** → **Add Context to Prompt** → **LLM** → **Final Answer**

- Retrieve Relevant Chunks
  - Similarity search
  - Top-K results
- Add Context to Prompt
  - Combine query + retrieved chunks into a prompt
- LLM
  - Generate answer using the context

2. Step by Step Flow

1. Load Documents → Read data from different sources.
2. Chunk Documents → Split into smaller pieces (chunks).
3. Create Embeddings → Convert chunks into vector representations.
4. Store in Vector DB → Save embeddings with metadata.
5. User Query → Convert query to embedding and search similar chunks.
6. Retrieve Context → Get top relevant chunks.
7. Generate Answer → Pass context + query to LLM and get final response.

3. Key Components

Component	Purpose
Documents	Source of knowledge
Chunking	Split text into smaller parts
Embedding Model	Convert text to vectors
Vector Database	Store & search vectors
Retriever	Find relevant chunks
LLM	Generate final answer

4. Important Notes

- RAG does not fine-tune the LLM.
- It retrieves relevant information at query time.
- The quality of retrieval directly impacts the quality of the answer.
- You can use your own data (private and up-to-date).

RAG for Developers

From Basics to Advanced

Documents, Chunking & Embeddings

1. What are Documents ?

- Documents are the source of knowledge that you want to use with RAG.
- They can be PDFs, text files, web pages, databases, Notion pages, etc.
- RAG works with unstructured or semi-structured data.

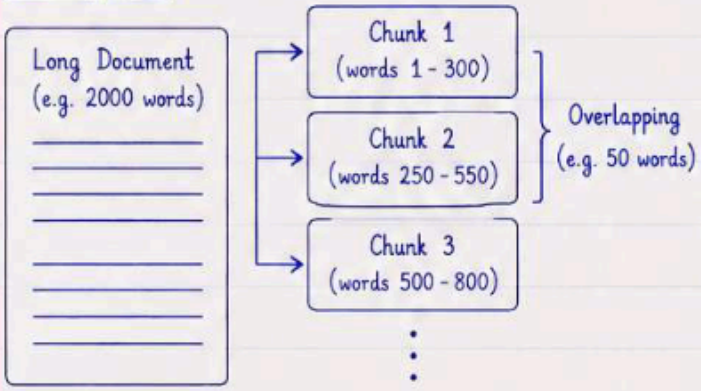
Examples of Documents

- Company policies (PDF)
- Product manuals
- Websites / blogs
- Support tickets
- Internal wikis / Notion
- Database records (exported as text)

2. Why Chunking is Required ?

- Documents can be long (hundreds of pages).
- LLMs and embedding models have a limit on input size.
- We split documents into smaller pieces (chunks) so that each chunk is meaningful and can be searched efficiently.

Example :



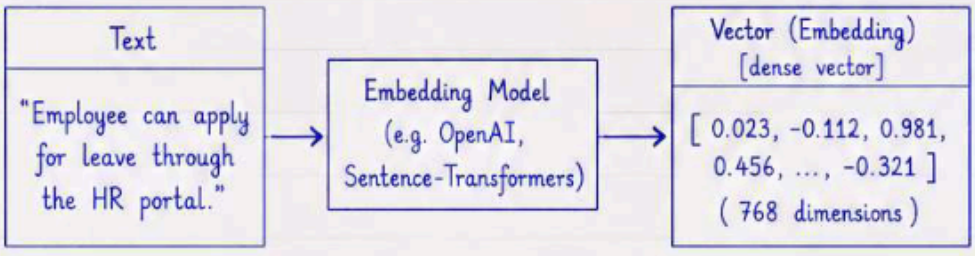
3. Chunking Parameters

- Chunk Size : Depends on use case (usually 200 - 1000 tokens)
- Overlap : Keeps context between chunks (usually 10% - 20%)
- Good chunks are self-contained and meaningful.

4. What are Embeddings ?

- Embeddings convert text into numerical vectors (dense representation).
- Similar text has similar vectors.
- These vectors capture the meaning of the text.
- We use embedding models like OpenAI, Sentence-Transformers, etc.

5. Example of Text to Embeddings



Key Takeaway :

Embeddings help us convert text into numbers so that we can search for similar content using vector similarity.

RAG for Developers

From Basics to Advanced

Vector Databases & Similarity Search

1. What is a Vector Database ?

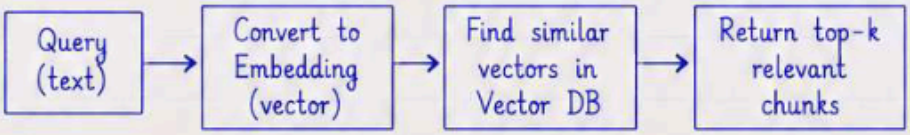
- A vector database stores and searches high-dimensional vectors (embeddings).
- It is optimized for similarity search instead of exact match.
- Used in RAG to store document embeddings and quickly find relevant chunks.

2. Why Not Traditional Databases ?

- Traditional databases work well for exact matches (e.g. SQL, keyword search).
- They don't understand the meaning or semantic similarity.
- Vector databases can find similar content even if the words are different.

Example :
Query: "How to reset my password?"
Can match with: "Steps to recover account access"
(even though words are different)

3. How Similarity Search Works ?



4. Cosine Similarity

- Most commonly used similarity metric in vector databases.
- Measures the angle between two vectors (higher value = more similar).
- Value ranges from -1 to 1 (usually 0 to 1 for embeddings).

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

where A and B are embedding vectors.

$\cos(\theta)$ close to 1 → more similar

$\cos(\theta)$ close to 0 → less similar

5. Metadata & Filtering

- Along with vectors, we can store metadata (e.g. file name, source, date, tags).
- Helps in filtering results (e.g. search only in a specific document or department).
- Improves relevance and control in RAG.

6. Popular Vector Databases

Vector DB	Key Points
Pinecone	Fully managed, easy to use
Chroma	Open source, lightweight
Weaviate	Open source, supports metadata
Qdrant	High performance, filtering
Milvus	Scalable, production ready
FAISS	Library (by Meta), in-memory

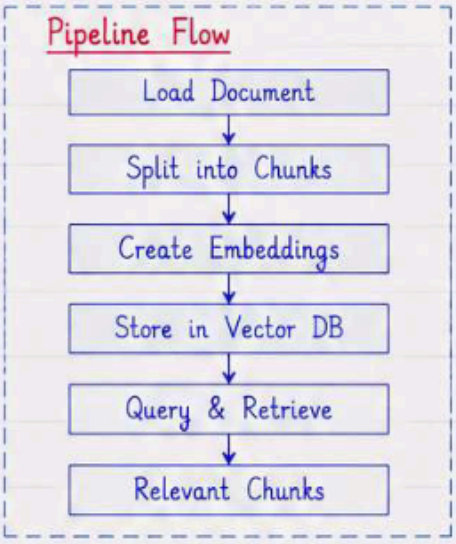
7. Key Takeaways

- Vector databases enable fast and accurate similarity search.
- Cosine similarity is widely used.
- Metadata filtering helps get more relevant results.
- Choose a vector database based on your use case (scale, cost, features).

Build a Basic RAG Pipeline

1. Overview

- In this page, we will build a simple end-to-end RAG pipeline.
- We will load a document, split it into chunks, create embeddings, store them in a vector database, and retrieve relevant chunks for a query.
- We will use Python with OpenAI embeddings and a vector database (Chroma).



2. Setup

- Install required libraries:

```
pip install openai langchain chromadb pypdf
```

- Set your OpenAI API key:

```
import os
os.environ["OPENAI_API_KEY"] = "your-api-key"
```

3. Load and Split Document

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load document
loader = PyPDFLoader("company_policy.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)

print(f"Total chunks: {len(chunks)}")
```

4. Create Embeddings

```
from langchain.embeddings import OpenAIEmbeddings

# Initialize embedding model
embeddings = OpenAIEmbeddings()

# Create embeddings for document chunks
docs = [chunk.page_content for chunk in chunks]
vectors = embeddings.embed_documents(docs)
print(f"Created {len(vectors)} embeddings")
```

5. Store in Vector Database (Chroma)

```
from langchain.vectorstores import Chroma

# Store embeddings in Chroma
db = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="rag_db"
)
print("Documents stored in Chroma!")
```

6. Query and Retrieve

```
# User query
query = "What is the leave policy?"

# Retrieve similar chunks
results = db.similarity_search(query, k=3)
for i, doc in enumerate(results):
    print(f"Chunk {i+1}: {doc.page_content}\n")
```

Key Takeaways

- We loaded a document and split it into chunks.
- Converted chunks into embeddings.
- Stored embeddings in a vector database.
- Retrieved relevant chunks for a user query.
- Next page we will see how to use these chunks with an LLM to generate the final answer.

RAG for Developers

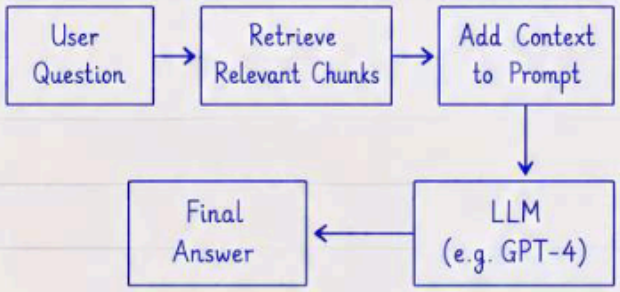
From Basics to Advanced

RAG with LLM + Prompt Engineering

1. How RAG Works with an LLM ?

- RAG retrieves relevant information from your documents.
- The retrieved context is added to the prompt.
- The LLM uses this context to generate a final answer.
- This helps in giving accurate, up-to-date and source-grounded responses.

RAG with LLM Flow



2. Prompt Structure for RAG

A good RAG prompt includes:

- System instruction (role of LLM)
- Retrieved context (from vector DB)
- User question
- Clear guidelines (e.g. don't use outside knowledge)

Example Prompt

You are a helpful assistant. Use only the following context to answer the question. If the answer is not in the context, say "I don't know".

Context: {retrieved_chunks}

Question: {user_question}

Answer:

3. Code Example (Using OpenAI)

```
from openai import OpenAI

client = OpenAI()

prompt = f"""
You are a helpful assistant. Use only the following
context to answer the question.

Context:
{context}

Question: {query}
Answer:
"""

response = client.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)
```

4. Controlling the LLM Response

- Instruct the LLM to use only the given context.
- Ask it to say "I don't know" if the answer is not present.
- You can also ask for sources (document name, page, etc.).
- Use a structured output format if needed (e.g. bullet points, table, etc.).

5. Example Response

User Question: What is the leave policy?

LLM Response:

According to the company policy, employees are entitled to 20 days of paid leave per year.
(Source: company_policy.pdf, Page 4)

6. Key Takeaways

- RAG + LLM = accurate and context-aware answers.
- Good prompt design is crucial.
- Always restrict the model to use only retrieved context.
- You can customize the response format as per your use case.

RAG for Developers

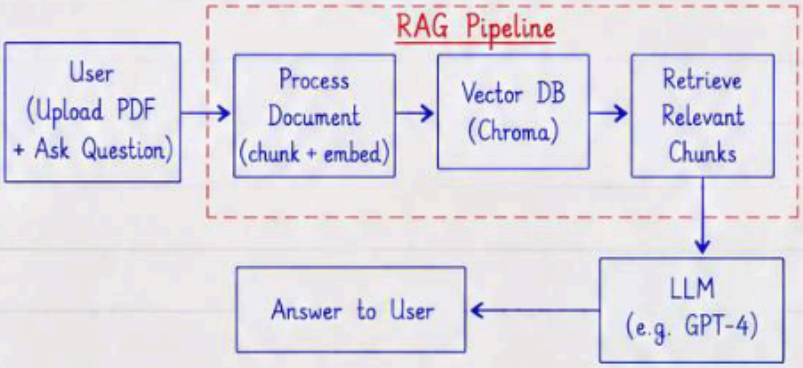
From Basics to Advanced

RAG Application Example: Chat with Your Documents

1. What are we building ?

- A simple application where user can upload a document (PDF).
- Ask questions about the document.
- Get accurate answers using RAG.
- We will use Python, OpenAI and Chroma (vector DB).

2. High Level Architecture



3. Upload and Load Document

```
# Load PDF document
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample_pdf")
documents = loader.load()

print(f"Loaded {len(documents)} pages")
```

4. Split, Embed and Store in Chroma

```
# Split into chunks
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)

# Create embeddings and store in Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
db = Chroma.from_documents(chunks, embeddings,
                           persist_directory="rag_db")

print("Documents stored in Chroma!")
```

5. Ask a Question (Retrieve + Generate)

```
# Create retriever
retriever = db.as_retriever(search_kwargs={"k":3})

# Create QA chain
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4"),
    retriever=retriever
)
query = "What is the leave policy?"
result = qa_chain.run(query)
print(result)
```

6. Sample Output

User: What is the leave policy?

Answer:
According to the company policy (Page 4), employees are entitled to 20 days of paid leave per year. Unused leaves can be carried forward for a maximum of 5 days.

7. Key Takeaways

- RAG allows you to build powerful document Q&A apps.
- We load, chunk, embed and store documents in a vector DB.
- Relevant chunks are retrieved and passed to an LLM.
- This approach works for PDFs, web pages, Notion docs, etc.
- You can build a simple UI later using Streamlit or FastAPI.

RAG for Developers

From Basics to Advanced

Improving RAG Quality & Retrieval

1. Why Quality Matters ?

- The final answer depends on the quality of retrieved chunks.
- If retrieval is poor, even the best LLM will give incorrect or incomplete answers.
- Improving retrieval = more accurate, relevant and reliable responses.

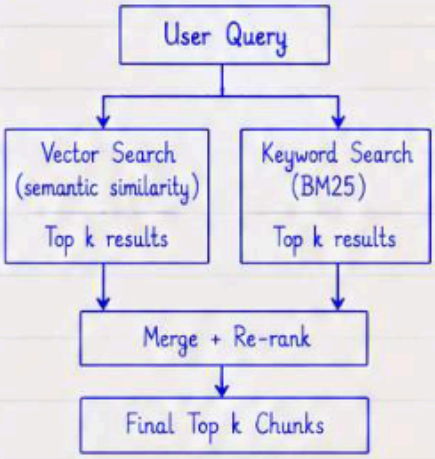
2. Common Problems in RAG

- Irrelevant chunks retrieved
- Important information missing
- Too much noise in context
- Chunks are too large or too small
- Similar meaning but different words not matched (e.g. "vacation" vs "leave")
- Outdated information
- No metadata or filtering

3. Techniques to Improve Retrieval

1. Better Chunking
   - Use meaningful chunk sizes (200 - 1000 tokens)
   - Keep context together (e.g. heading + content)
2. Overlapping Chunks
   - Use 10% - 20% overlap to avoid missing information
3. Metadata & Filtering
   - Add metadata (source, date, tags, department)
   - Filter results based on user intent
4. Better Embedding Models
   - Use high-quality models (e.g. text-embedding-3-large)
   - Consider domain-specific models if needed
5. Hybrid Search
   - Combine vector search + keyword search (BM25)
   - Helpful for exact terms, IDs, or rare words
6. Re-ranking
   - Use a re-ranker model to score and sort retrieved chunks (e.g. Cohere Rerank, OpenAI, etc.)
7. Query Transformation
   - Rewrite or expand the user query
   - Handle synonyms, abbreviations, and follow-up questions

4. Hybrid Search (Vector + Keyword)



Why Hybrid ?

- Vector search understands meaning.
- Keyword search helps with exact matches (e.g. IDs, error codes).
- Combining both improves accuracy and relevance.

5. Query Transformation Example



6. Re-ranking Example

Chunk	Initial Score	Re-rank Score	Final Rank
Chunk A	0.82	0.91	1
Chunk B	0.78	0.85	2
Chunk C	0.80	0.80	3
Chunk D	0.76	0.60	4

Re-ranking uses a separate model to evaluate how relevant each chunk is for the query.

7. Key Takeaways

- Good retrieval is critical for RAG success.
- Use proper chunking, metadata, and filtering.
- Hybrid search and re-ranking improve relevance.
- Transforming queries helps handle real user questions.
- Keep experimenting with different techniques based on your use case.

RAG for Developers

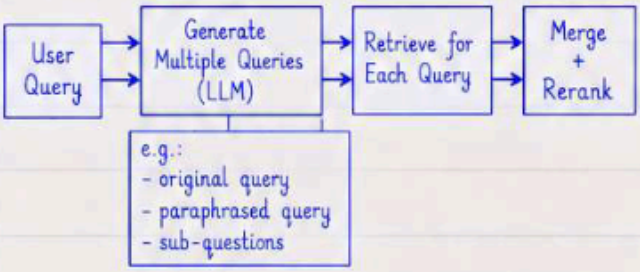
From Basics to Advanced

Advanced RAG: Multi-Query, Reranking, Context Compression & Agentic RAG

1. Multi-Query Retrieval

- Generate multiple versions of the user query to capture different intents and phrasings.
- Helps improve recall (especially for complex questions).
- Use LLM to generate alternative queries (e.g. paraphrases, sub-questions).
- Retrieve and merge results from all queries.

Multi-Query Flow



2. Reranking

- Initial retrieval (vector search) may return many relevant but not well-ordered results.
- Use a reranker model to reorder chunks based on deeper semantic understanding.
- Common models: Cohere Rerank, OpenAI Rerank, Sentence Transformers.
- Improves precision and relevance of top results.

Example (Cohere Rerank)

```
from cohere import Client
co = Client(api_key="your-api-key")
results = co.rerank(
    model="rerank-english-v3.0",
    query=query,
    documents=documents,
    top_n=5
)
for r in results:
    print(r.index, r.relevance_score, documents[r.index])
```

3. Context Compression

- Retrieved chunks can be large and contain irrelevant information.
- Use an LLM to compress or extract only relevant parts from the chunks.
- Techniques: summarization, context filtering, extracting key sentences.
- Helps reduce token usage and improves answer quality.

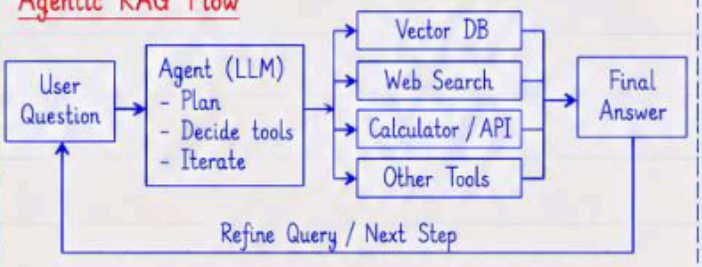
Example (Compression Prompt)

```
Prompt:
"""
Given the following document, extract only the information relevant to the question: {query}
Document: {chunk}
Relevant information:
"""
```

4. Agentic RAG (Tool-using RAG)

- For complex questions, a single retrieval may not be enough.
- Use an agent (LLM) to plan, retrieve, iterate and use tools (e.g. search, calculator, database) if needed.
- Agent decides what to do next (e.g. reformulate query, retrieve more docs, use external API).
- Useful for multi-hop questions and knowledge-intensive tasks.

Agentic RAG Flow



5. When to Use These Techniques ?

Technique	Use Case
Multi-Query	Complex or ambiguous user queries
Reranking	Improve relevance of retrieved results
Context Compression	Large chunks, irrelevant information
Agentic RAG	Multi-hop questions, tool usage, advanced use cases

6. Key Takeaways

- Multi-query improves recall.
- Reranking improves precision.
- Context compression saves tokens and improves quality.
- Agentic RAG enables complex, real-world use cases.
- Combine these techniques based on your application needs.

RAG for Developers

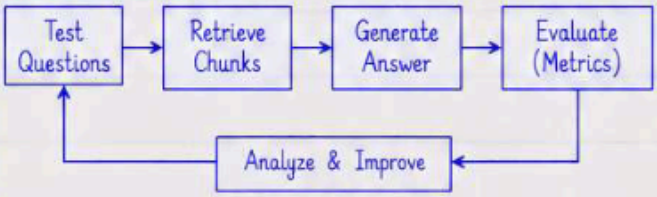
From Basics to Advanced

Production RAG: Evaluation, Security, Cost, Latency & Best Practices

1. Evaluation - How to Measure RAG ?

- Evaluation helps ensure the answers are accurate, relevant and grounded in the retrieved context.
- Common metrics:
  - Retrieval Recall - relevant chunks are retrieved.
  - Context Precision - retrieved chunks are relevant.
  - Answer Correctness - final answer is correct.
  - Hallucination Rate - model generates unsupported info.
  - User Satisfaction - real user feedback.

Evaluation Flow



2. Security & Data Privacy

- Documents may contain sensitive information (e.g. PII, financial data, internal policies).
- Always follow data security and compliance guidelines.
- Use access control and document-level permissions.
- Avoid sending sensitive data to public LLMs.
- Consider using private LLMs (Azure OpenAI, on-prem models).
- Apply data masking or redaction if needed.

Security Best Practices

- Authenticate and authorize users.
- Restrict access to specific documents.
- Mask sensitive data (PII, passwords, account numbers).
- Use encrypted storage (vector DB).
- Follow compliance standards (GDPR, SOC2, HIPAA, etc.).
- Log and monitor usage.

3. Cost Optimization

- Embedding and LLM calls can be expensive.
- Optimize to reduce cost without impacting quality.

Ways to Reduce Cost

- Use smaller/cheaper embedding models (e.g. text-embedding-3-small)
- Cache embeddings and LLM responses.
- Use chunk filtering to reduce number of tokens.
- Retrieve top-k relevant chunks (not too many).
- Use shorter prompts and context compression.
- Use a cheaper model for simple queries and a powerful model only when needed.

4. Latency & Performance

- Users expect fast and real-time answers.
- Optimize each step in the pipeline.

Techniques to Improve Latency

- Use efficient vector search (e.g. HNSW index in Chroma).
- Retrieve only top-k chunks (e.g. 3-5).
- Use smaller, low-latency LLMs for simple queries.
- Implement caching (retrieved chunks + final answers).
- Run embedding and retrieval in parallel where possible.
- Stream the LLM response to the user.
- Monitor latency at each stage (retrieval, LLM, total).

5. Monitoring & Logging

- Monitoring helps detect issues and improve over time.
- Log user queries, retrieved chunks, model responses and errors.

What to Monitor ?

- Number of requests
- Retrieval hit rate
- Answer correctness (manual/automatic)
- Latency and errors
- User feedback (thumbs up/down)
- Cost per request
- Model performance and drift over time

6. Best Practices for Production RAG

1. Start with a simple pipeline and iterate.
2. Use good chunking, metadata and hybrid search.
3. Always provide source references in the answer.
4. Handle edge cases (no relevant docs, out-of-scope questions).
5. Keep the system secure and compliant.
6. Optimize for cost and latency.
7. Continuously evaluate and monitor.
8. Collect user feedback and improve.
9. Keep experimenting with new models and techniques.
10. Document your architecture and maintain the system.

Final Takeaways

- RAG is a powerful way to build real-world, domain-specific AI applications.
- Quality, security, cost and latency are critical for production.
- Keep iterating, measure performance, and focus on user value.
- Start simple, build, test, and improve continuously.